CONFIDENTIAL INTERNAL USE ONLY

Linux Networking Evolution

Linux v3.0 から **7.x** まで **– 15** 年間でカーネルネットワークは何が変わったか

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

本講演の構成

Part I

中心命題

複数の execution / memory-ownership model の共存と explicit contracts

Part II

4 つの architecture era

外部の圧力に対して何を解こうとしたか

Part III

長期 lineage

PERFORMANCE / PROGRAMMABILITY / MEMORY / CONTROL PLANE + cross-cutting domain

Part IV

Driver framework

driver-local convention が共通 contract へ移った経緯

Part V

Observability

複雑化した path を説明可能にする evidence layer

Synthesis

まとめと未完の仕事

接続点の整理と contract がまだ届いていない箇所

対象範囲: wired / host networking。RDMA・Wi-Fi 全般・QUIC・security protocol の網羅的な歴史は扱わない。

Red Hat

Part I – 中心命題

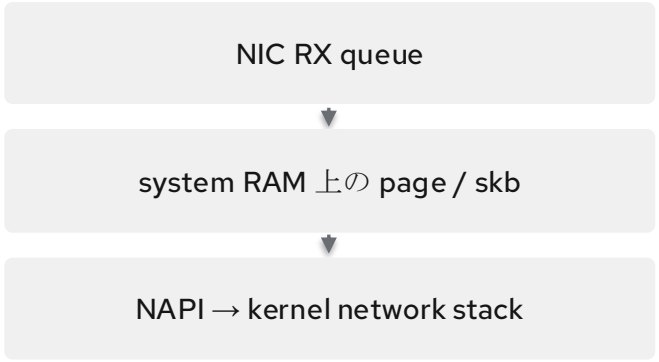
Linux networking は何が変わったのか
「高速化」でも「skb path の置き換え」でもない

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

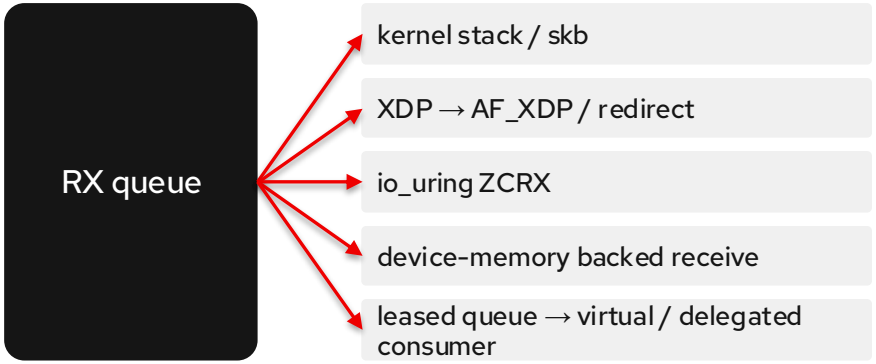
skb path は残り、複数の execution / memory model が共存する

2011 年頃 (v3.0)



RX queue と system RAM / skb / stack の対応を暗黙に想定しやすかった

2026 年



同じ「RX queue」が、通常 stack / XDP / io_uring / device memory / virtual device への入口になり得る

Takeaway queue identity・memory provider・consumer・lifetime・synchronization を明示し、複数モデルを同じ device 上で選択・共存できるようにしたことが中心命題

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

変化は 6 つの軸で並行して進んだ

PERFORMANCE

queueing / pacing / aggregation / zero-copy

implicit queueing → explicit accounting / multi-layer scheduling / larger processing units

PROGRAMMABILITY

BPF / XDP / socket / protocol / qdisc extension points

fixed path → verified extension points → reusable programmable infrastructure

MEMORY

page_pool / netmem / memory providers / device memory

driver-local page handling → common lifetime / provider / assignment model

CONTROL PLANE

Netlink object model / YNL / RTNL scope reduction

embedded / global state → reusable objects / schema / narrower lock scope

OBSERVABILITY

typed tracing / drop reason / queue-NAPI-memory identity

opaque outcome → typed reason / identity / correlation

DRIVER FRAMEWORK

switchdev / devlink / phylink / DIM / common driver contracts

driver-local convention → reusable kernel framework / device-wide object

TRANSPORT / VIRTUAL-OVERLAY / SECURITY は 7 番目以降の軸ではなく、複数の軸を横断する cross-cutting domain

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

共存を可能にした mechanism: explicit contracts

object
再利用可能な named object

accounting
queued / completed bytes 等の計量

API
verified / schema-driven interface

assignment
queue・memory を誰が使うか

lifetime
allocation / recycle / release

synchronization
lock scope・HW state 同期

Milestone	以前の状態	主に明示化したもの	Contract form	Era
DQL/BQL	driver-local TX ring tuning	queued / completed byte accounting	accounting	1
bpf() / maps / verifier	fixed kernel behavior	verified program + map interface	API	1
switchdev / devlink	vendor-local forwarding / device state	forwarding / device / resource objects	object	2
page_pool	driver-local page recycle	allocation / recycle / DMA lifetime	lifetime	2
AF_XDP	kernel-centric queue / buffer	queue / UMEM binding	assignment + lifetime	2
nexthop objects	route に埋め込まれた nexthop	reusable nexthop object / group	object	3
YNL	handwritten Netlink family	machine-readable Netlink schema	API	4
Device Memory TCP	payload は system RAM 前提	non-system-RAM memory in networking path	assignment + lifetime	4
per-netns RTNL	global RTNL scope	namespace-scoped synchronization	synchronization	4
RX HW queue leasing	HW queue は physical netdev に固定	hardware queue assignment / leasing	assignment	4

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

Snapshot: v3.0 / v5.0 / 2026

観点	v3.0 前後	v5.0 前後 (中間観測点)	2026
PERFORMANCE	driver-local tuning が大きい	BQL / TSQ / fq / pacing が定着	multi-layer control + cake_mq / scalable TX
PROGRAMMABILITY	classic BPF 中心	XDP / AF_XDP・TC BPF	struct_ops / TCX / BPF qdisc / netkit
MEMORY	page-centric driver-local recycle	page_pool が利用可能	netmem / memory providers / device memory
DRIVER FRAMEWORK	net_device 中心	switchdev / devlink / phylink	queue/NAPI objects・YNL-described APIs
CONTROL PLANE	global RTNL 依存が大きい	global RTNL が依然中心	per-netns / fine-grained / RTNL-less paths
OBSERVABILITY	logs / counters / tracing の個別利用	BPF / BTF ecosystem が拡大	typed drop reason + queue/NAPI/memory identity

v5.0 は第 2 の起点ではなく、XDP / AF_XDP、switchdev / devlink、page_pool など programmable / offload / memory infrastructure がすでに成立した中間観測点として置く。

Red Hat

Part II – 4 つの architecture era

kernel major version ではなく、外部の圧力に対して
何を解こうとしたかで区切る

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

外部の圧力から **architecture response** へ

Era 1 ～2014

Queue & scalability foundation

圧力
10/40GbE・multicore・container・datacenter latency

代表 milestone
BQL, CoDel/fq_codel, TSQ, pacing, namespace/setns, VXLAN, nftables, eBPF foundation

Era 2 2015–2018

Programmable datapath

圧力
cloud networking・high packet rate・SmartNIC/offload・kernel-bypass pressure

代表 milestone
TC BPF, XDP, cgroup/LWT BPF, SOCKMAP, switchdev, devlink, AF_XDP, page_pool

Era 3 2019–2022

Programmability becomes infrastructure

圧力
hyperscale operations・protocol experimentation・100GbE 級・typed tooling

代表 milestone
BTF ecosystem, struct_ops, SK_LOOKUP, MPTCP, BIG TCP, io_uring networking, drop reason, devlink health

Era 4 2023～

Explicit placement & scoped control

圧力
accelerator/device memory・400GbE 級・heterogeneous execution・global-lock scalability

代表 milestone
netmem, memory providers, Device Memory TCP, netdev-genl queue/NAPI, queue leasing, per-netns RTNL, BPF qdisc, YNL

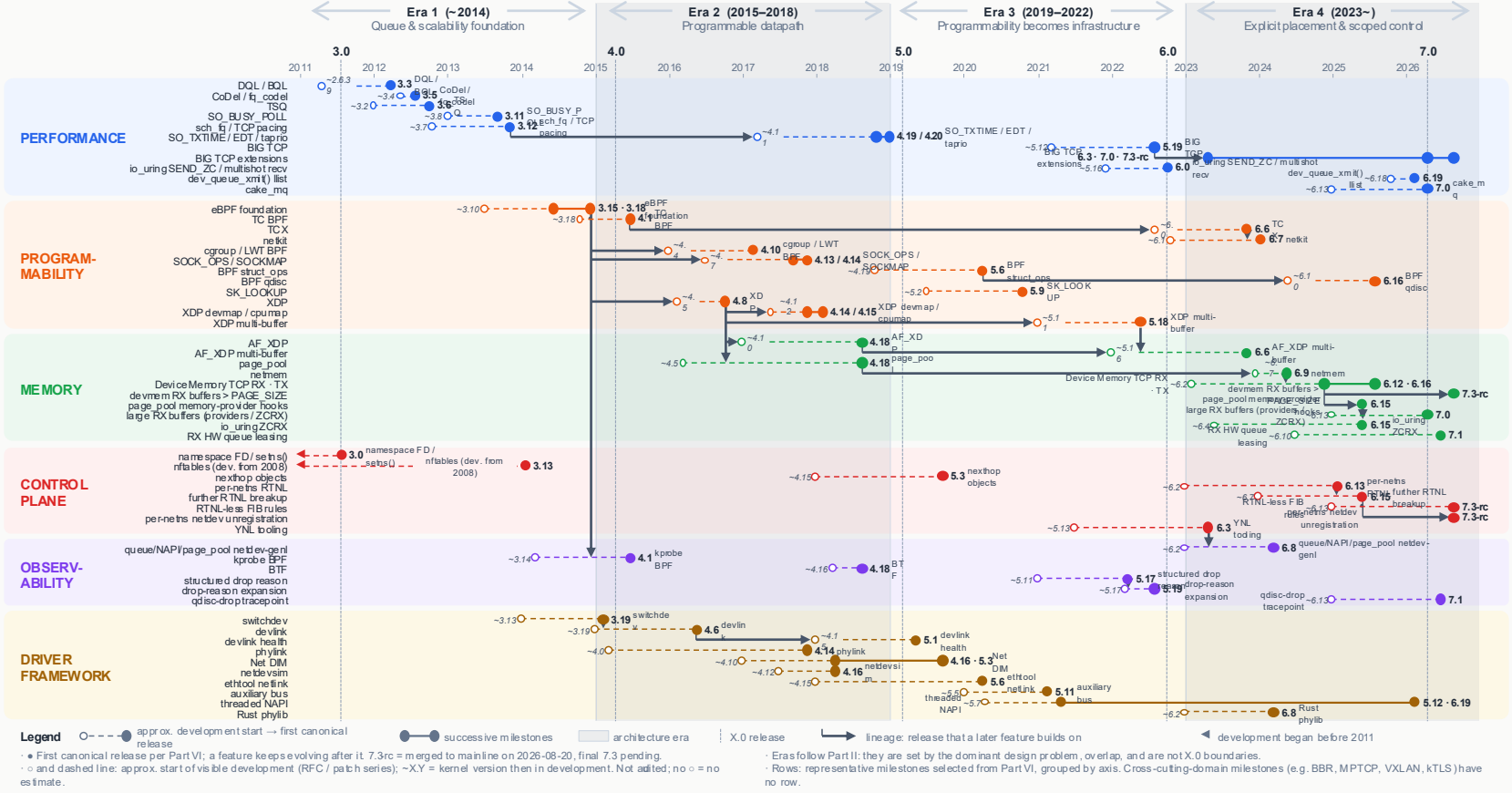
Era 間で変わった設計上の問い

How much should we queue? → Where can we program the datapath? → How do those mechanisms become reusable infrastructure? → How explicitly can placement, lifetime and synchronization be controlled?

Red Hat

Explicit resource / control contracts let execution and memory models coexist (2011–2026)

Linux networking milestones by architecture axis, set against the four architecture eras and X.0 releases · canonical edition r52



· ● First canonical release per Part VI; a feature keeps evolving after it. 7.3rc = merged to mainline on 2026-08-20, final 7.3 pending.
· ○ and dashed line: approx. start of visible development (RFC / patch series); ~X.Y = kernel version then in development. Not added; no ○ = no estimate.

· Eras follow Part II: they are set by the dominant design problem, overlap, and are not X.0 boundaries.
· Rows: representative milestones selected from Part VI, grouped by axis. Cross-cutting-domain milestones (e.g. BBR, MPTCP, VXLAN, kTLS) have no row.

Part III – 長期 lineage

release 順ではなく、同じ設計課題が
長期間にどう変化したかを追う

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

PERFORMANCE

Queueing: 単一 qdisc から multi-layer control へ

socket / transport	**TSQ (3.6) · TCP pacing (3.12) · BBR (4.9) · EDT (4.20)** socket が作る burst を制限し、「いつ送るか」を扱う
qdisc / scheduler	**CoDel · fq_codel (3.5) · sch_fq (3.12) · CAKE (4.19) · taprio (4.20) · cake_mq (7.0)** sojourn time を feedback に delay / fairness / shaping、time-aware scheduling
driver / NIC	**DQL/BQL (3.3) · SO_REUSEPORT (3.9) · SO_BUSY_POLL (3.11) · dev_queue_xmit() llist (6.19)** hardware queue の outstanding bytes を accounting、CPU/queue placement、TX serialization 縮小

- CoDel は queue length ではなく sojourn time を制御する転換点。fq_codel で flow isolation + AQM へ
- ETF / SO_TXTIME / taprio は AQM と目的が異なり、explicit transmission-time scheduling まで広げた
- mac80211 airtime / AQL (5.1 / 5.5) は bytes ではなく無線 medium 時間を resource として扱う – hidden queue を accounting 可能にする

Takeaway queue occupancy と completion を accounting / API として明示し、各 layer が backpressure と scheduling を独立に制御できるようになった

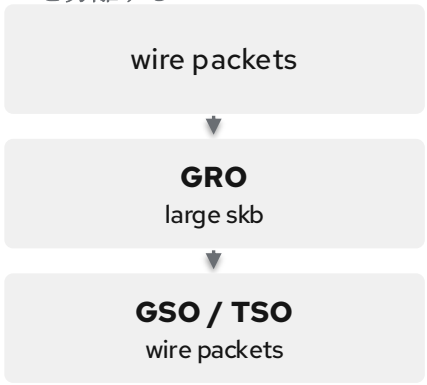
Red Hat

CONFIDENTIAL INTERNAL USE ONLY

PERFORMANCE

Packet aggregation: GRO/GSO から BIG TCP へ

stack 内部の processing unit と wire packet を分離する



BIG TCP lineage – 64 KiB 制約の緩和

5.19	**IPv6 BIG TCP** gro_max_size / gso_max_size > 65536
6.3	**IPv4 BIG TCP**
7.0	**IPv6 BIG TCP without synthetic HBH jumbo header**
7.3-rc	**BIG TCP over VXLAN / GENEVE** mainline; final pending

- BIG TCP は wire MTU を巨大化する機能ではなく、kernel 内部の packet-processing unit を大きくする機能 – jumbo frame の別名ではない
- UDP GRO/GSO (5.0) や PPPoE GRO/GSO も同じ軸で per-packet cost を減らすが、BIG TCP の派生ではない

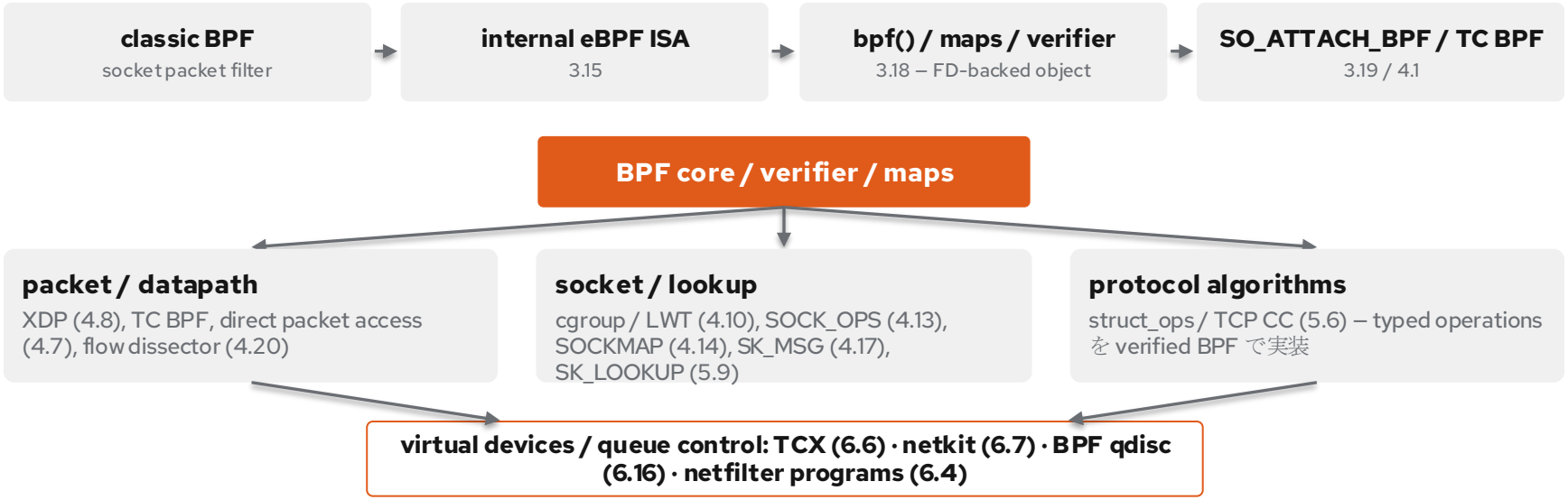
Takeaway aggregation は explicit contract の系列ではなく、1回の stack traversal で扱う processing unit を拡大する並行系列

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

PROGRAMMABILITY

BPF: packet filter から stack extension へ



Takeaway extension point、program type、map、typed operations を API / object として明示 – language の置換ではなく verified execution contract と attachment model の一般化

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

PROGRAMMABILITY

XDP / AF_XDP: placement と ownership

XDP at driver RX (4.8) – skb 構築前の RX buffer で verified BPF を実行

XDP_PASS	XDP_DROP	XDP_TX	XDP_REDIRECT
通常 stack へ	早期破棄	受信 device から送り返し	DEVMAP (4.14) / CPUMAP (4.15) / XSK map

- 高速性は「BPF だから」ではなく、skb allocation 以前という placement と redirect 先を明示する object / API から来る
- multi-buffer / frags (5.18) で non-linear packet を扱える一般的な RX representation へ
- XDP metadata / RX hints: HW timestamp・RSS hash・VLAN 等を packet bytes とは別 contract で BPF へ渡す

AF_XDP (4.18) – RX queue ↔ UMEM / userspace buffer の binding



- zero-copy mode では UMEM frame と RX queue を直接結び付けるため、queue binding と memory ownership が correctness の一部になる
- AF_XDP multi-buffer (6.6)、virtio-net AF_XDP RX ZC (6.11)

Takeaway verified execution point に加え、redirect target・RX queue・UMEM ownership を object / assignment として明示し、kernel path と userspace path の共存を可能にした

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

MEMORY

Packet memory: page_pool → netmem → providers

page_pool
4.18 – RX memory lifecycle の共通化

→

netmem
6.9 – struct page と packet memory を同一視しない

→

Device Memory TCP RX / TX
6.12 / 6.16 – system RAM を経由しない path

→

memory-provider hooks
6.15 – devmem TCP と io_uring ZCRX を共通 contract へ

Layer	役割	代表
page_pool	RX fast path の allocator / recycler – allocation・DMA-aware recycling・in-flight lifecycle	page_pool (4.18), introspection (6.8)
netmem	memory representation – 「page であること」ではなく access / lifetime operation で memory を扱う	netmem_ref (6.9)
memory provider	allocation / lifetime provider – page allocator・device memory・io_uring memory を同じ RX lifecycle へ	memory_provider_ops (6.15)
queue binding / leasing	assignment – どの RX queue がその memory domain を使うか	devmem binding, ZCRX, RX HW queue leasing (7.1)

Takeaway packet buffer の lifetime・provider・queue assignment を driver-private convention から共通 contract へ移した

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

MEMORY

Zero-copy と io_uring networking

TCP MSG_ZEROCOPY
4.14 – send-side copy avoidance

→

TCP_ZEROCOPY_RECEIVE
4.18 – RX payload を mapping で渡す

→

io_uring SEND_ZC / multishot
6.0 – registered resources + async completion

→

io_uring ZCRX
6.15 – RX queue と userspace memory を直結

traditional

NIC → system RAM → CPU / copy / mapping → GPU / accelerator

device-memory path (Device Memory TCP)

NIC → device memory → GPU / accelerator

- operation completion と memory lifetime completion の分離 – MSG_ZEROCOPY は error queue、SEND_ZC は zero-copy completion で「buffer 再利用可」を別途通知
- TCP_ZEROCOPY_RECEIVE は transparent optimization ではなく、memory layout と socket consumption semantics を API へ露出する contract (recv_skip_hint)
- ZCRX では RX queue / page_pool / memory provider と registered userspace memory を接続 – queue ownership と buffer lifetime が io_uring object model の外部条件になる

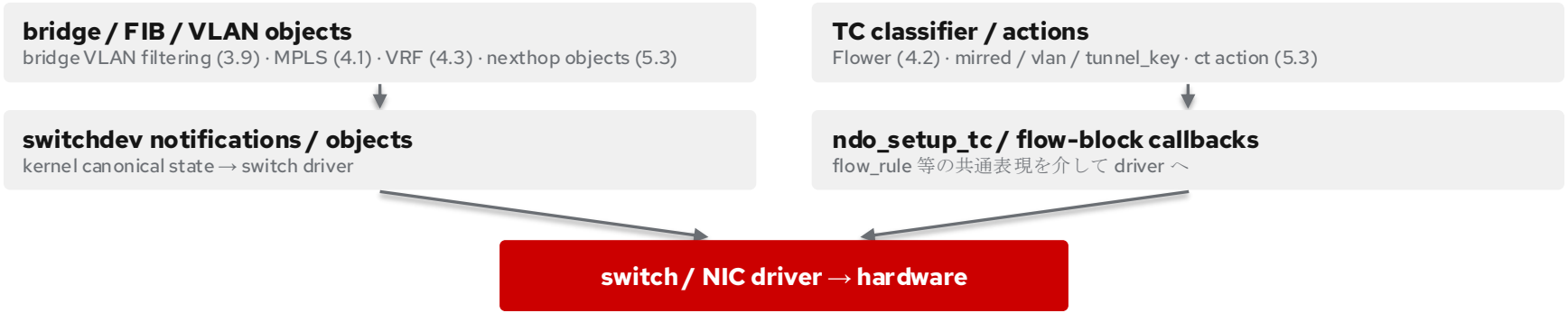
Takeaway socket I/O を submission / completion と registered resources に分解し、ZCRX では queue と buffer / memory-provider の関係まで明示する方向へ

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

CONTROL PLANE

Routing / TC / offload: 2 つの control path



- switchdev と TC offload を一本の経路として扱わない – kernel API としては並行する経路が driver / hardware で合流し得る
- nexthop object (5.3): route に埋め込まれた nexthop を ID 付き reusable object へ。ECMP group 変更が「多数の route 書き換え」から「共有 object の更新」へ。ASIC の adjacency table とも対応しやすい
- offload は TC rule を hardware-native rule に置換するのではなく、kernel canonical state + optional hardware execution – 何が offload 可能か・一部 unsupported 時の扱い・stats の読み出し元が contract になる

Takeaway forwarding semantics を kernel の canonical state として保ちながら、bridge / FIB / TC policy を software または hardware execution へ写像できるようになった

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

CONTROL PLANE

Routing object・YNL・RTNL scope 縮小

routing object

route → embedded nexthop

↓

IPv4 route-cache removal (3.6)

↓

route → reusable nexthop object → group / resilient group (5.3)

Netlink API – YNL (6.3)

YAML specification

- UAPI
- policy
- generated helper
- documentation
- userspace client

netdev family が queue / NAPI / page_pool を object として公開。WireGuard も YNL-described に (6.19)

RTNL scope

global RTNL

↓

unlocked operations / RCU readers

↓

per-netns RTNL (6.13～)

↓

further RTNL breakup (6.15)

↓

RTNL-less FIB rule updates / per-netns netdev unregistration (7.3-rc)

Takeaway Netlink message format より shared mutable state の保護範囲を縮小することが主眼 – routing object と synchronization scope を明示し、global RTNL dependency を縮小

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

CONTROL PLANE

netfilter: rule engine から fast path へ

iptables / ebtables / arptables
family ごとの xtables

→

nftables core
3.13 – expression / VM, sets / maps, transactional update

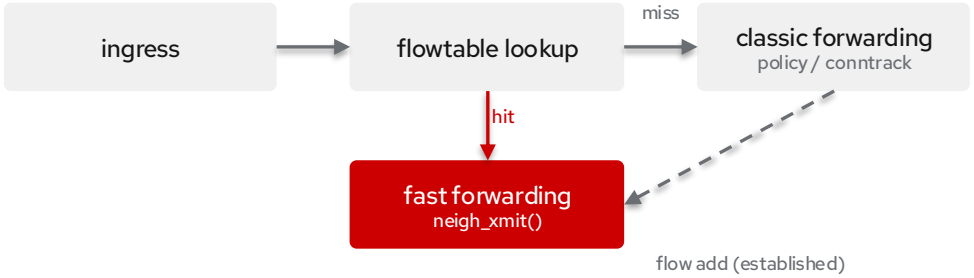
→

ingress hook / netdev family
4.2 (egress 5.16)

→

flowtable
4.16 software fast path → 5.5 HW offload

flowtable – policy と fast path の分離



- conntrack は置換されていない – state / NAT / mark / lifetime を nftables・TC ct・BPF kfunc・flowtable が共有する substrate
- HW offload (5.5) は flow を flow_rule として driver へ写像。投入は非同期で、完了まで software path を通る packet もある
- BPF / XDP も nftables の置換ではない – bpfilter は 6.8 で削除、定着したのは conntrack kfunc・BPF_PROG_TYPE_NETFILTER (6.4) の相互乗り入れ

Takeaway policy representation・transactional update・shared flow state・fast-path eligibility・execution placement が明示化され、slow / fast / hardware path が共存

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

Virtual networking: datapath と queue assignment

network namespace / setns()	VXLAN · Geneve	ipvlan	vDPA bus	netkit	RX HW queue leasing
3.0	3.7 · 3.18	3.19	5.7	6.7	7.1

VM datapath

virtio-net ‖ vhost ‖ vDPA ‖ SR-IOV / VFIO

- AF_XDP zero-copy と相互作用
- virtio-net AF_XDP RX ZC (6.11)
- representor: VF/SF port を host 側 net_device として表現

container datapath

veth ‖ netkit / BPF-native path

- netkit は veth の単純な後継ではない
- peer device 内部に BPF execution point
- per-CPU backlog overhead の削減

queue / zero-copy assignment

AF_XDP / io_uring ↔ RX HW queue leasing

- virtual-device queue と physical queue の対応を明示
- queue-bound API を container / virtual device 側から利用可能に
- TX queue leasing は未 merge

Takeaway full kernel bypass ではなく、kernel の control / protection model を維持しながら data movement と datapath overhead を減らす選択肢を増やした

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

TCP / UDP / transport: protocol semantics 自体の並行進化

Theme	Representative milestones	何を変えたか
connection establishment	TFO client / server (3.6 / 3.7)	handshake と application data の境界、early data
congestion control / feedback	DCTCP (3.18) ‖ BBR (4.9) ‖ PLB (6.2) ‖ AccECN (6.18 / 7.0)	ECN feedback、bandwidth / RTT model、ECMP path 再選択、CE feedback の情報量
multipath	MPTCP (5.6〜)	1 つの logical connection の下に subflow / address / path-manager state
authentication	TCP-AO (6.7)	MD5 signature の後継、key-management model
socket / lifecycle	TCP_REPAIR、TSQ、pacing、SOCK_OPS (4.13)	checkpoint / restore、socket 側 buffering 制限、rate-controlled transmission

MPTCP object model

byte stream → MPTCP connection / data sequence space → subflow A / B → path A / B

Path Manager: endpoint object, ADD_ADDR / REMOVE_ADDR, subflow create / destroy

- TCP 上で使われるという理由だけで memory / aggregation / programmability の milestone をここへ再収容しない – BIG TCP、Device Memory TCP、struct_ops CC、ZCRX は別の lineage
- UDP: MSG_ZEROCOPY (5.0) と GRO/GSO は protocol semantics の変更ではなく PERFORMANCE milestone

Takeaway transport は explicit-contract thesis だけでは説明しない – connection / congestion / path / feedback semantics 自体を更新する並行系列

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

SECURITY: semantics / state と execution placement の分離

kTLS
TLS record state on TCP socket

- kTLS TX (4.13) – handshake は userspace、record datapath は kernel
- kTLS RX (4.17)
- TLS device offload TX / RX (4.18 / 4.19)
- TLS_SW と TLS_HW は同じ API に対する execution placement の違い

XFRM / IPsec
SA + policy = kernel canonical state

- crypto offload API – xfrmdev_ops (4.12): NIC が encrypt / decrypt
- packet offload (6.2): SA + policy を NIC と同期し、encapsulation / policy execution まで NIC へ
- rekey / lifetime / policy synchronization が contract の中心

WireGuard
peer / key / allowed-IP + netdevice

- 5.6 – L3 secure tunnel を通常の network device driver として実装
- RTNL / genl / UDP tunnel API / GRO-GSO / NAPI へ統合
- XFRM の次世代でも offload lineage でもない – SECURITY と VIRTUAL / OVERLAY の交差点

Takeaway 暗号方式の変遷ではなく、security state / policy を kernel が保持したまま software・accelerator・NIC のどこで実行するかを明示的な contract で選べるようになった

Red Hat

Part IV – Network device driver framework

driver-local な実装知識が、
共通 framework / object / API へどう引き上げられたか

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

DRIVER FRAMEWORK

driver-local convention から共通 contract へ



queue accounting	**DQL / BQL** 3.3	accounting (backpressure)
link topology	**phylink** 4.14	API / synchronization
switch datapath	**switchdev** 3.19	object / synchronization
device control / health	**devlink / devlink health** 4.6 / 5.1	object / API / lifetime
interrupt moderation	**DIM** 4.16 → 5.3	measurement → policy → driver setting
packet memory	**page_pool → netmem** 4.18+	lifetime / object
queue / NAPI identity	**netdev-genl** 6.8+	object / assignment
NIC configuration	**ethtool netlink / YNL** 5.6	structured API

Takeaway driver-private だった state / resource / algorithm boundary を kernel 共通 contract へ引き上げ、複数 driver・hardware・execution model から再利用可能にした

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

DRIVER FRAMEWORK

Modern driver contracts

devlink – device-wide control object

PCI / physical device → devlink

- ports / representor
- resources (TCAM 等の有限 resource)
- parameters / regions
- health reporters: diagnose / dump / recover (5.1)

datapath 自体は提供しない。switchdev / TC offload が「どこで forwarding を実行するか」、devlink は resource / configuration / health lifecycle

netdev-genl – queue / NAPI identity (6.8+)

physical netdev

- NAPI object
- RX queue object
- TX queue object
- page_pool identity / stats (6.8)

AF_XDP、ZCRX、memory providers、queue leasing のように特定 queue へ resource を binding する API が増えると、queue を安定して指す control-plane identity が必要になる

execution context / capability

- threaded NAPI (5.12): softirq ではなく kthread で poll – execution context の選択肢
- threaded busy-poll (6.19): NAPI 単位で thread / affinity / scheduler policy を割り当てる contract
- XDP features: binary な「XDP 対応」から redirect / ndo_xmit / zero-copy 等を machine-readable に公開
- Rust PHY abstractions (6.8): C subsystem の ownership / lifetime rule を Rust type system へ写像
- netdevsim (4.16) / auxiliary bus (5.11): API 検証と driver composition

Takeaway control plane は capability + resource identity + assignment を別々に問い合わせられるようになった

Red Hat

Part V – Observability / Explainability

programmable / offloaded / zero-copy path を
operationally explainable にする evidence layer

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

OBSERVABILITY

Observability: 並行する evidence を相関する

Primitive	主に提供するもの	境界 / 注意点
BTF (4.18)	kernel type / field metadata – CO-RE relocation	runtime event や packet trajectory 自体は記録しない
eBPF tracing (kprobe BPF 4.1)	attach point での event / state	coverage、attach point、権限、overhead に依存
skb_drop_reason (5.17 / 5.19)	対応箇所での structured drop reason	全 drop path が必ず reason を付与するわけではない
timestamping	特定地点での時刻情報 (SO_TIMESTAMPING / PHC)	clock、取得地点、HW/SW semantics に依存
netdev-genl (6.8) / qdisc-drop tracepoint (7.1)	queue / NAPI / page_pool の identity・state・stats	可視化できることと ownership / configuration を変更できることは別

Tool の case study

pwru function-level trajectory を広く探索する debugger

Retis networking event / skb metadata / OVS-OVN context を意味的に enrich して相関

- cross-layer packet journey は単一 primitive の機能ではなく、tool が identifier・event・timestamp・metadata を相関して推論する目標
- Observability は 7 番目の contract form ではなく、resource / control contract と runtime behavior を検証する evidence layer
- IOAM は kernel 内部を外から観測するのではなく、network path 上で telemetry を packet 自身へ運ぶ

Takeaway 「counter → drop reason」の一本道ではなく、counter・tracepoint・structured reason・identity が併存し、問いに応じて異なる evidence を提供する

Red Hat

Synthesis – thesis への回帰と未完の仕事

何が明示されたから、何が共存できたか

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

contract が共存を可能にした接続点

明示化された対象	Contract の lineage	その接続点で共存できるようになったもの
queue occupancy	DQL/BQL → TSQ / sch_fq / pacing	socket・qdisc・driver が backlog を別々の層で制御する multi-layer control
execution point	bpf() / maps / verifier → XDP・TC・cgroup・socket hooks → struct_ops / TCX / BPF qdisc	built-in stack と verified program が同じ packet・socket・algorithm を扱う
packet memory	page_pool → netmem → memory providers	system-RAM page、userspace buffer、device memory が同じ RX architecture に接続
queue identity	AF_XDP queue / UMEM binding → netdev-genl queue/NAPI → RX HW queue leasing	NIC queue を kernel stack、AF_XDP、ZCRX、device-memory path 、virtual device が使い分ける
forwarding / device state	switchdev・devlink → representor / TC offload → XFRM packet offload	canonical semantics を保持しながら software path と hardware path を選べる
control-plane state	nexthop objects → YNL → per-netns RTNL	reusable object / machine-readable API / narrower lock scope で configuration を並行化
policy / flow state	nftables → conntrack consumers → flowtable / HW offload	transactional policy と shared state を保ちながら slow / fast / hardware path を選択
security state / policy	kTLS / XFRM / WireGuard	kernel-visible security state を保ちつつ software / NIC execution または netdevice integration を選択

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

contract に還元しない並行系列

(2) transport / protocol semantics

TFO · DCTCP · BBR · MPTCP · PLB · AccECN · TCP-AO

接続確立、輻輳制御、multipath、feedback、authentication の semantics 自体を更新

(3) processing unit の拡大

GRO/GSO · UDP GRO · BIG TCP

1 回の処理で扱う単位を拡大 – contract 化ではなく processing-unit expansion

(4) implementation scalability

dev_queue_xmit() llist (6.19) · route-cache removal (3.6)

contract を変えずに実装を速くする変更

(1) resource / control contract の明示化 – 本書の中心 thesis。どの系列を説明しているかを限定した上で成り立つ

- routing / forwarding は第 5 の並行系列ではなく、CONTROL PLANE と VIRTUAL / OVERLAY、PROGRAMMABILITY / DRIVER FRAMEWORK が交差する connection point
- これらは thesis の例外ではなく、別の説明を要する並行系列 – thesis は未来予測ではなく既存 milestone を横断した説明モデル

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

未完の仕事 – explicit contracts がまだ届いていない箇所

TX 側の queue assignment

RX HW queue leasing は 7.1 に入ったが TX queue leasing は未 merge。assignment contract は RX 側に偏っている

global synchronization の縮小

per-netns RTNL は一括置換ではなく複数 release にわたる migration。7.3-rc の RTNL-less FIB rule / netdev unregistration は途中経過

異種 memory の一般化

netmem (6.9) と provider hooks (6.15) は boundary を確定。残る課題はより多くの driver / memory type の provider-aware 化

execution placement の共通化

switchdev path と TC offload path は並行経路のまま。software / hardware のどちらで実行するかを扱う単一の contract はない

observability の coverage

drop reason はすべての drop path に付くわけではなく、cross-layer packet journey は tool 側の推論に依存

Rust

PHY / core abstraction が中心で、一般的な high-performance NIC driver が Rust へ移行した段階ではない

将来の release で評価が変わり得るため、確定史ではなく open questions として扱う

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

まとめ

1

置き換えではなく共存

system-RAM + skb path は残り、AF_XDP・device memory・io_uring ZCRX・programmable / offloaded path が同じ device 上で選択・共存できる architecture へ拡張した

2

それを可能にしたのは **explicit contracts**

driver-private / implicit / global だった state を object・accounting・API・assignment・lifetime・synchronization として明示した箇所が、新しい path の接続点になった

3

並行系列と **evidence layer**

transport semantics、processing unit、implementation scalability は別の系列。observability は contract と runtime behavior を検証する evidence layer

Era 1: How much should we queue? → Era 4: How explicitly can placement, lifetime and synchronization be controlled?

Red Hat

CONFIDENTIAL INTERNAL USE ONLY

Thank you

release attribution の正本は Part VI、commit-level provenance は Part VII (48 anchors, upstream 照合済み)

Red Hat